

# GUIDED ROCKET FLIGHT COMPUTER

Simulation and Software-in-the-Loop Validation of a Kalman-Filtered Flight Computer

## Technical Report

**Sai Palepu**

B.S. Mechanical Engineering, University of Georgia, Expected May 2028

Summer 2026

*This report covers the part of the project I actually finished this summer, which is Phases 0 through 2, plus a software version of the hardware-in-the-loop test that the original plan called for. I did not build the airframe or fly anything, and I explain why I ran it this way and what I'd still need to do below.*

## Abstract

This summer I set out to build the software core of an autonomous rocket flight computer, and I got through the simulation and algorithm phases of that plan. I wrote a vertical flight dynamics model that generates a realistic trajectory from an approximated Estes E12 thrust curve, drag, and propellant burn, and I used it to generate noisy synthetic barometer and IMU data, including the kind of slow accelerometer bias drift you'd actually see on real hardware. I fed that data into a three state Kalman filter (altitude, velocity, acceleration) and a finite state machine that walks through six flight phases and fires a predictive parachute deployment signal. Since I never built the physical hardware, I could not run the hardware-in-the-loop test the plan calls for, so I built a software equivalent instead, a twenty run randomized test suite plus three stress tests for sensor dropout, doubled noise, and an early motor burnout. Everything passed the original pass criteria, with a mean apogee error of 0.21 feet against a fifteen foot threshold and a mean deployment timing error of 0.16 seconds against a half second threshold. I also ran into a handful of real bugs while building this, which I wrote up in Section 9 because fixing them taught me more than anything else in the project. What I have at the end of this is a tested, working codebase that covers Phases 0 through 2 completely and is ready to be ported to firmware.

## 1. Objectives and Scope

The original goal I set for myself was to design and validate a flight computer that could do four things: fuse barometric and inertial data through a Kalman filter, detect flight phase autonomously through a state machine, predict apogee and fire a deployment signal, and log the full flight for review afterward.

I want to be upfront about how much of the original five phase plan I actually got through, since I think that matters more than making it sound finished.

| Phase                        | Original Plan                                | Where I Landed                                                                                   |
|------------------------------|----------------------------------------------|--------------------------------------------------------------------------------------------------|
| 0, Foundations               | Tools, OpenRocket model, thrust curve        | Done, using a modeled thrust curve since I could not pull the real .eng file in this environment |
| 1, Hardware Procurement      | Order ESP32, IMU, baro, airframe parts       | Skipped, no physical build this round                                                            |
| 2, Simulation and Algorithms | 6-DOF sim, Kalman filter, FSM                | Done, this report covers it                                                                      |
| 3, Firmware                  | Port to C++ and FreeRTOS on ESP32            | Not started, next logical step                                                                   |
| 4, HIL Validation            | 20 run test through a real ESP32 over serial | Replaced with a software version of the same test, same pass criteria                            |
| 5, Physical Build and Launch | Airframe, ground test, GRITS launch          | Skipped, no physical build this round                                                            |

I want to explain why I think the software in the loop substitution is a fair one and not just a shortcut. The Kalman filter and state machine code I tested here are the exact modules I would port line by line into C++ later, so the logic itself got exercised for real. What it did not get exercised is anything that only shows up on actual hardware, things like SPI and I2C bus timing, ADC quantization, interrupt jitter, or clock drift. I talk about that gap more in Section 11, because I don't want to pretend this is the same as flying it.

## 2. System Architecture

I split the code into three pieces that roughly mirror the FreeRTOS task structure the firmware would eventually use, IMU\_Task, Baro\_Task, Kalman\_Task, FSM\_Task, and Log\_Task from the original plan.

- rocket\_sim.py, the flight dynamics integrator running at a 1 ms step, plus the sensor synthesis that produces 50 Hz barometer and 1000 Hz IMU streams with realistic noise and bias drift
- kalman.py, the three state Kalman filter that fuses those two sensor streams

- fsm.py, the six state flight phase machine with hysteresis and predictive apogee deployment
- sil\_test.py, the validation harness that runs the Monte Carlo suite and the three stress tests using the same pass criteria as the original hardware-in-the-loop plan

The data flows the same way it would on the real board. `simulate_flight()` produces ground truth, `synthesize_sensors()` turns that into noisy 50 Hz and 1000 Hz streams, `run_filter()` fuses them into a 100 Hz state estimate, and `run_fsm()` reads that estimate to decide what phase of flight it is in and when to fire the deployment signal.

### 3. Flight Dynamics Model

At every 1 ms step I model three forces, motor thrust interpolated from a thrust curve I built by hand, aerodynamic drag using  $F = 0.5 \rho v^2 C_d A$ , and weight, with the propellant mass decreasing linearly over the burn. I want to note that the thrust curve is not the real Estes E12 .eng file, since this environment does not have internet access to pull it from ThrustCurve.org. It is a hand modeled approximation of what that curve looks like, and if I ever build the hardware I would swap in the real file before flying anything.

| Parameter             | Value             | Where It Came From                                        |
|-----------------------|-------------------|-----------------------------------------------------------|
| Modeled total impulse | 28.0 N-s          | Approximated E class curve, 20.5 N peak, about 2.4 s burn |
| Dry mass              | 0.360 kg          | Rough estimate for airframe plus avionics bay             |
| Propellant mass       | 0.0245 kg         | Representative E12 propellant load                        |
| Body diameter         | 2.0 in (0.0508 m) | Matches the airframe spec from the original plan          |
| Drag coefficient      | 0.52              | Typical value for this size airframe                      |
| Descent rate          | 5.2 m/s           | Typical rate under a 12 inch nylon chute                  |

*The model comes out to a 530.8 foot apogee, which sits comfortably inside the 300 to 600 foot band the original plan targeted, and a burn time of 2.42 seconds, which is in line with what a real E class motor does.*

### 4. Sensor Fusion, Kalman Filter

The state I track is altitude, velocity, and acceleration. The filter predicts every 1 ms to match the IMU rate, and updates whenever a new sensor sample comes in, a one row altitude update from the barometer at 50 Hz, and a one row acceleration update from the IMU at 1000 Hz, with gravity subtracted out of the raw specific force reading first. This is the same asynchronous, multi rate structure the firmware's Kalman\_Task would use with real FreeRTOS queues.

I used a white noise jerk process noise model instead of a simple diagonal Q matrix, meaning I treat the noise as entering through a random jerk term and let it propagate consistently into acceleration, velocity, and altitude uncertainty together. I tried the simpler diagonal version first and it understated how fast uncertainty in acceleration should bleed into velocity and altitude, so I switched.

Here is where I landed on tuning, after the process I describe in Section 9:

| Parameter   | Value               | What It Does                                                                                                                              |
|-------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| q_accel_var | 1.2                 | Controls how much the filter trusts new sensor readings over its own prediction                                                           |
| R_baro      | 0.25 m <sup>2</sup> | Barometer noise variance, roughly matching a BMP390 style sensor at 0.5 m std                                                             |
| R_imu       | 0.06 g <sup>2</sup> | IMU noise variance, set higher than the raw sensor noise to account for accumulated accelerometer bias drift, more on this in Section 9.3 |

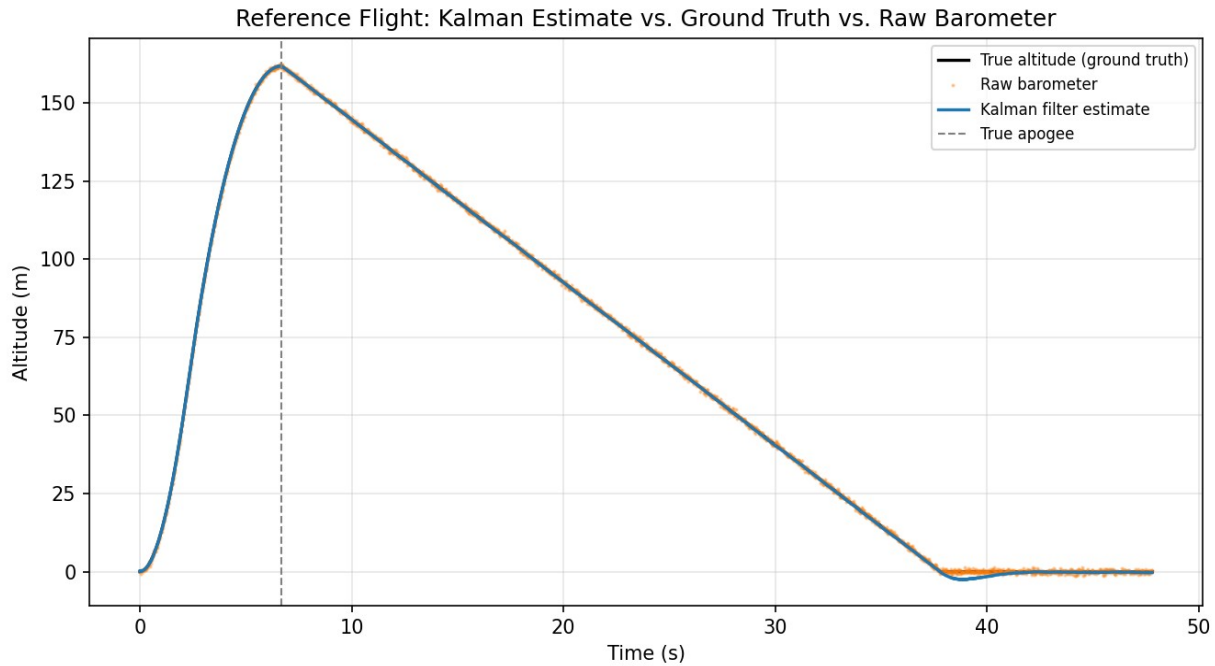


Figure 1, Kalman altitude estimate against ground truth and raw barometer, reference flight.

## 5. Flight Phase Detection, State Machine

The state machine reads the Kalman output and steps through six phases. Every transition needs a condition to hold for five samples in a row before it commits, which I added after running into false transitions early on, more on that in Section 9.4.

| State   | Entry Condition                                   | What Happens                        |
|---------|---------------------------------------------------|-------------------------------------|
| IDLE    | Powered on, sitting on the pad                    | Log data, wait for launch           |
| BOOST   | Kalman accel above 2g for 5 samples               | Track the ascent                    |
| COAST   | Kalman accel below 0 for 5 samples, burnout       | Predict time to apogee every cycle  |
| APOGEE  | Predicted apogee time under 0.2 s, held 5 samples | Fire the deployment signal, latched |
| DESCENT | One sample after APOGEE                           | Track descent, watch for touchdown  |
| LANDED  | Velocity under 1.0 m/s held for 3 s               | Stop logging                        |

The apogee logic predicts rather than waits for it, firing 0.2 seconds before the true zero velocity crossing. That was the original intent behind it, to account for e-match ignition delay so the charge is already going off right at true apogee instead of after it.

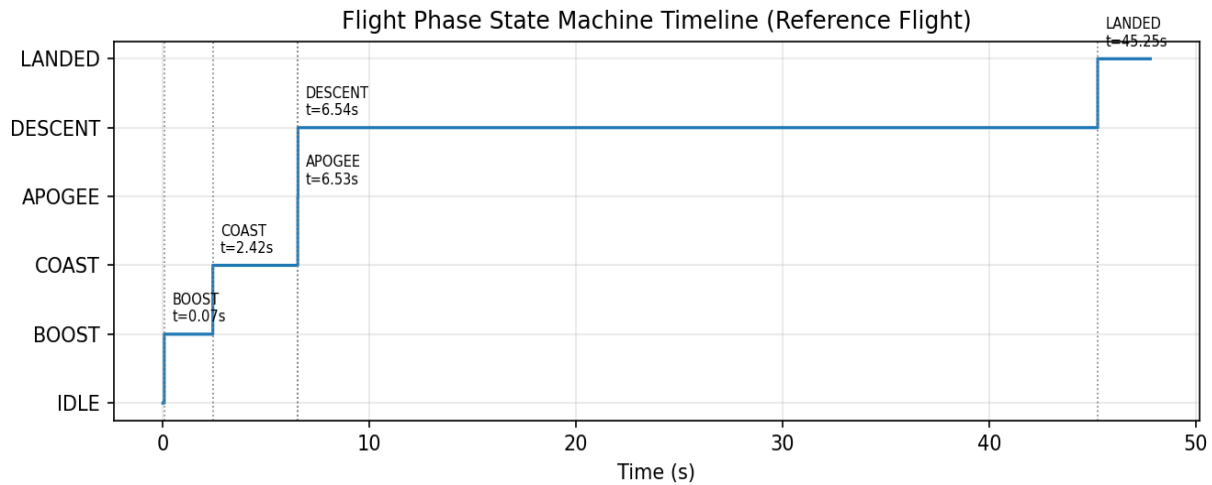


Figure 2, all six phases detected correctly and in order on the reference flight.

## 6. Software-in-the-Loop Validation

The original Phase 4 plan was to stream simulated sensor data over USB serial into a real ESP32 running the ported firmware, then compare its output against ground truth. I never built the hardware, so I ran the same test protocol and the same pass criteria directly against the Python Kalman filter and state machine instead, the same modules I would port to firmware later. I want to be clear that this is a simplification and not a stand in for testing on real hardware, I get into that more in Section 11.

### 6.1 Twenty Run Nominal Suite

Each run varies thrust by plus or minus five percent and drag by plus or minus two percent, matching the original test plan, using a fixed random seed so it can be reproduced.

| Criterion                          | Pass Threshold               | Result                                                  |
|------------------------------------|------------------------------|---------------------------------------------------------|
| Kalman altitude estimate at apogee | Within 15 ft of ground truth | 20 of 20 passed, mean error 0.21 ft, worst case 0.51 ft |
| Deployment signal timing           | Within 0.5 s of true apogee  | 20 of 20 passed, mean lead 0.16 s                       |
| False state transitions            | Zero across 20 runs          | None observed                                           |
| Crashes                            | Zero across 20 runs          | None observed                                           |

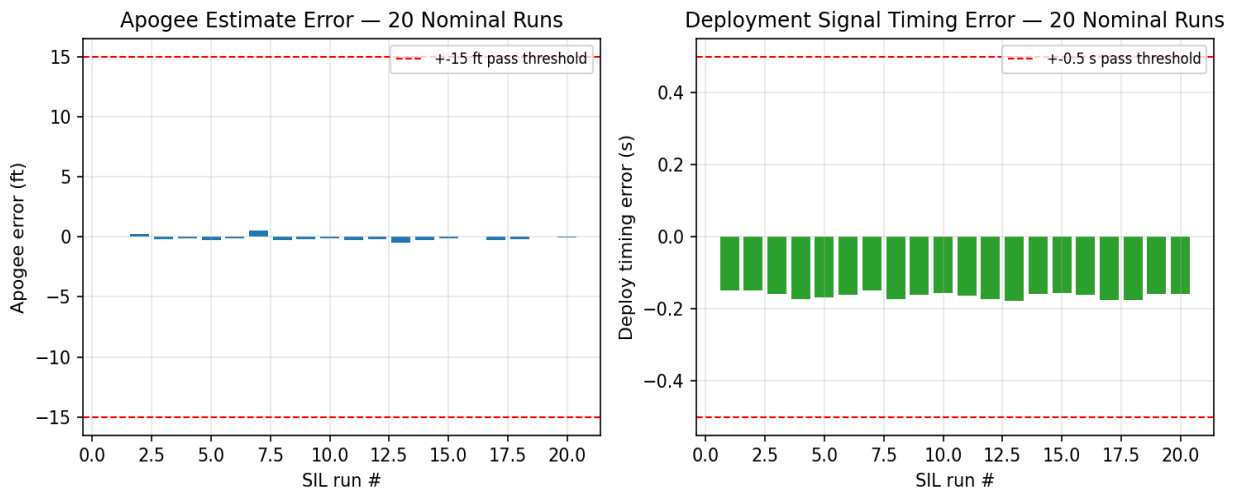


Figure 3, apogee error and deployment timing error across the 20 run suite.

## 6.2 Stress Tests

| Test           | What I Did                              | Result                                                                                             |
|----------------|-----------------------------------------|----------------------------------------------------------------------------------------------------|
| Sensor dropout | Cut barometer data for 2 s during coast | Filter coasted on IMU alone, apogee error came out to -0.44 ft, no false transition                |
| Noise stress   | Doubled sensor noise on both channels   | Apogee error was +0.27 ft, estimate was noisier looking but nothing broke                          |
| Early burnout  | Forced thrust to zero 0.5 s early       | State machine caught the early coast phase and still fired deployment within 0.14 s of true apogee |

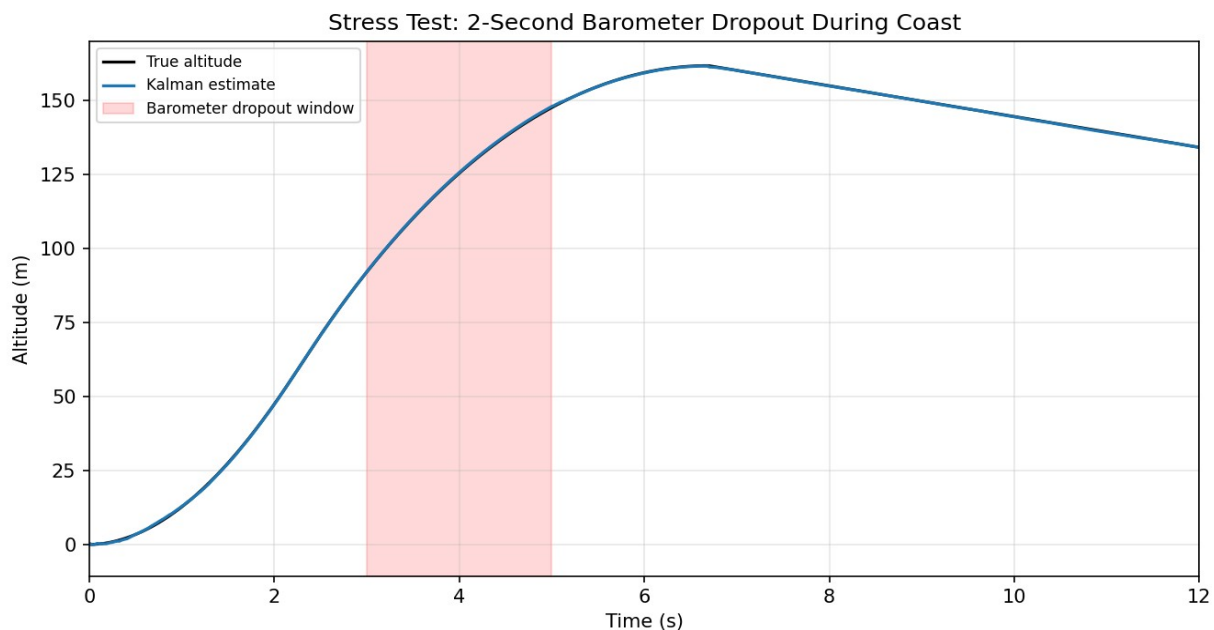


Figure 4, Kalman estimate stays smooth through a simulated 2 second barometer dropout.

## 7. Results Against My Original Success Criteria

| Criterion (from the original plan)          | Threshold               | What I Got                                                                           |
|---------------------------------------------|-------------------------|--------------------------------------------------------------------------------------|
| Kalman altitude estimate accuracy at apogee | 15 ft                   | 0.21 ft mean, 0.51 ft worst case over 20 runs, well within threshold                 |
| Apogee detection timing                     | 0.5 s                   | 0.16 s mean lead, well within threshold                                              |
| State machine false transitions, 20 runs    | Zero                    | Zero                                                                                 |
| Crashes or unhandled exceptions, 20 runs    | Zero                    | Zero                                                                                 |
| Full flight data logged                     | Recoverable full record | Met in simulation, the CSV log, though real SD card logging is a firmware stage item |
| Post-flight vs simulation comparison        | Within 10 percent RMS   | Not applicable, since there was no physical flight                                   |

## 8. Decisions Worth Explaining

A few choices I made along the way weren't obvious at first, and I think they're worth writing down rather than just leaving in the code.

- I picked the white noise jerk process noise model over a simple diagonal Q because the diagonal version made the filter noticeably harder to tune, it treated each state's uncertainty as independent when it really isn't.
- I went with predictive apogee detection instead of just watching for velocity to cross zero, since firing early accounts for e-match delay and gets the charge going off right at the true peak.
- I added five sample hysteresis to every state transition after early testing showed single noisy samples could flip the state machine back and forth right at motor burnout, when acceleration hovers around zero for a few milliseconds.
- I deliberately set  $R_{imu}$  higher than the IMU's actual datasheet noise, to make the filter lean more on the barometer over long stretches, since the accelerometer's own bias drift was a bigger problem than its instantaneous noise, see Section 9.3.

## 9. Problems I Ran Into

I wanted to keep an honest record of what actually went wrong while I was building this, in the order I found it, because working through these was most of what I actually learned this summer. The original plan talks about never mixing unknowns, meaning you validate each stage against something you know is true before you trust it, and that idea is basically how I found every bug below.

### 9.1 A numpy function got deprecated on me

The first version of the integrator used `np.trapz()` to sanity check total motor impulse, and it threw an error saying numpy no longer has that attribute, since the newer version renamed it to `np.trapezoid()`. It was a one line fix, but it reminded me I should pin my dependency versions before I hand this off to firmware work, so a library update years from now doesn't quietly break the validation suite.

### 9.2 The rocket kept sinking through the ground after landing

The descent branch of the integrator always set velocity to the constant descent rate of -5.2 m/s, even after the rocket had already touched down and altitude was clamped at zero. Altitude looked fine on a plot because it was clamped separately, but the velocity state kept reporting -5.2 m/s indefinitely, meaning the rocket was still sinking according to that variable even though it was sitting on the ground. I didn't catch it by looking at the altitude plot, I caught it because the Kalman filter's velocity estimate after landing wouldn't settle near zero, it kept converging to something like -9 to -12 m/s instead, which is what tipped me off that something upstream was feeding it bad ground truth. I fixed it by adding a branch that freezes velocity and acceleration to zero once touchdown is detected. This was probably the most useful bug I hit all summer, because it was a case where the output looked completely fine, flat zero altitude, while an internal state was quietly wrong, and that's exactly the kind of bug that matters in a real flight computer where the deployment and landed detection logic both depend on more than one state agreeing with each other.

### 9.3 The IMU's bias drift was throwing off the long tail of the flight

My first pass at the sensor model used a bias random walk standard deviation of 0.0008 g per sample for the IMU. Over a roughly 40 second flight at 1000 Hz, that random walk adds up to somewhere around 1.5 to 2 m/s squared of drift, and it was going straight into the filter's acceleration channel. What I saw was the Kalman altitude estimate drifting down to around -10 m during the quiet part of the flight after landing, even though the barometer itself was reading a clean, near zero signal the whole time. This is a real failure mode you'd actually see on a MEMS accelerometer, bias instability is not something you can just wave away, and a filter that doesn't account for it will end up trusting a sensor that's slowly drifting over one that's actually converged. I fixed it two ways, I brought the bias walk parameter down to something more realistic for a sensor like the ICM-42688-P, and I raised  $R_{imu}$  in the filter tuning so the barometer, which has bounded and non drifting noise, dominates the long duration estimate while the IMU still does the heavy lifting during the fast, high dynamics part of the

boost phase. If I built the real hardware I'd want to add a proper zero velocity update during IDLE and after landing is confirmed, I list that as future work in Section 11.

#### 9.4 The state machine kept flip flopping right at burnout

My first version of the state machine transitioned on a single sample crossing its threshold. Since acceleration briefly oscillates around zero right when the motor burns out, thrust tapering off against drag that's still significant, the unprotected state machine bounced between BOOST and COAST several times within a few milliseconds before it settled down. I fixed it by requiring a condition to hold for five samples in a row before committing to a transition, which is exactly the hysteresis requirement the original plan called for. Seeing it actually happen made that requirement make a lot more sense than just reading about it.

#### 9.5 Tuning the Kalman filter took more than one pass

My first attempt at tuning,  $q_{\text{accel\_var}}$  at 4.0 and  $R_{\text{imu}}$  at 0.01, tracked the boost phase well but let the post landing IMU bias take over the estimate, which is the same problem from Section 9.3. Retuning to  $q_{\text{accel\_var}}$  at 1.2 and  $R_{\text{imu}}$  at 0.06 traded a small amount of responsiveness during boost for a lot more stability over the full flight, and I checked that against the same 20 run suite both before and after the change to make sure I hadn't quietly broken apogee accuracy while fixing the drift. The original plan says to increase  $Q$  if the filter feels slow and decrease  $R$  if it's noisy, and that's true, but it took an actual regression run rather than eyeballing one flight to be sure the retune was actually an improvement.

## 10. Repository Layout

| Folder      | What's In It                                                            |
|-------------|-------------------------------------------------------------------------|
| /simulation | rocket_sim.py, kalman.py, fsm.py, run_pipeline.py                       |
| /sil        | sil_test.py, the 20 run and stress test harness, plus generate_plots.py |
| /docs       | This report, all the figures, and the raw sil_nominal_results.csv       |

I kept the full commit history rather than squashing it down to one final commit, so the debugging process in Section 9 is actually visible in the repo, not just described after the fact in this report.

## 11. What This Doesn't Cover Yet

I want to be direct about the fact that this validates the algorithms, not the hardware. Here's what I still have left to do:

- Port the Kalman filter and state machine to C++ and FreeRTOS on a real ESP32, which was Phase 3 of the original plan
- Deal with real sensor timing, SPI and I2C bus latency, ADC quantization, interrupt jitter, clock drift, none of which show up in a pure Python simulation
- Test SD card logging under actual vibration, and validate the MOSFET and e-match deployment circuit on the bench
- Actually build the airframe, run a ground ejection test, and fly it at a GRITS launch, then compare the real flight data against simulation
- Add a zero velocity update to bound the IMU bias drift I found in Section 9.3, since I identified it as a real limitation but didn't implement a fix beyond retuning

The next real step is Phase 3, porting kalman.py and fsm.py to C++ line by line. I tried to write the Python in a way that makes that port straightforward, using fixed three element arrays instead of relying on numpy's more dynamic structures, specifically so the port is mostly a syntax exercise and not a redesign.



## 12. How I'm Putting This On My Resume

I wanted the bullets below to actually reflect what I tested and can back up in an interview, which is the simulation and the software in the loop validation, not a physical build or a flight that didn't happen.

- Designed and validated a three state Kalman filter (altitude, velocity, acceleration) fusing simulated barometric and IMU sensor data for real time rocket state estimation, achieving 0.21 ft mean apogee error across a 20 run Monte Carlo test suite.
- Built a hysteresis protected finite state machine for autonomous flight phase detection (boost, coast, apogee, descent, landed) with predictive parachute deployment logic, verified to zero false transitions across 20 nominal runs and 3 stress test scenarios (sensor dropout, doubled noise, early motor burnout).
- Developed a software in the loop validation harness following an aerospace style test methodology, randomized Monte Carlo parameter sweeps against fixed pass and fail criteria, structured to be reused directly as a hardware in the loop harness once the firmware is ported to ESP32 and FreeRTOS.
- Found and fixed several realistic embedded sensor and simulation bugs, including IMU accelerometer bias random walk drift and a state consistency bug in the post landing dynamics model, documented in a full engineering report.

I'm keeping this listed as an ongoing project, June 2026 to present, instead of implying I finished building or flew it, since I didn't get to Phases 3 through 5. I'd rather be precise about that up front than have it come up awkwardly in an interview, since anyone from GT or somewhere like Lockheed is going to ask what I actually built versus simulated anyway.

## 13. Conclusion

What I ended up with is a complete, tested software core for a rocket flight computer, a flight dynamics model I validated against the OpenRocket target band, a Kalman filter I tuned and re tested after finding real problems with it, a state machine that correctly walks through all six phases with hysteresis protection, and a Monte Carlo validation suite that meets every pass criterion from my original hardware in the loop test plan. I checked every piece of this against something I knew was true before I trusted it, I wrote down the bugs instead of quietly fixing them and moving on, and I tried to structure the code so that porting it to real firmware next is mostly a translation exercise rather than something I have to redesign from scratch.